



How Does Shape-Constrained Regularization Impact the Performance of EWMA, ARIMA, & GARCH Models in Multi-Horizon Volatility Forecasting for Financial Markets?

Nischal Adhikari

Institution: Pennsylvania State University

Introduction

- **Objective:** Explore & compare EWMA, ARIMA, & GARCH models
 - Evaluate the impact of shape-constrained regularization
- Volatility forecasting → risk management, pricing derivatives, & portfolio optimization.
 - Constraints (monotonicity) → robustness & interpretability
- **Key Questions:**
 - How do these constrained / unconstrained models perform?
 - Does monotonicity improve forecast reliability or introduce bias?

[illegible]

Volatility Forecasting Models

Aspect	EWMA	ARIMA	GARCH
Complexity	Simple & Fast	Moderate	High
Strengths	Ease to implement, responsive to shocks	Captures autoregression & trends	Captures volatility clustering
Weaknesses	Ignores long-term patterns	Struggles with non-linear patterns	Computationally intensive
Forecasting Horizon	Short-term	Short-to-medium-term	Short-to-long-term
Best Use Case	Stable markets, short-term forecasting	Stationary time series w/ trends	Financial data w/ clustering effects

Model	Formula	Key Parameters
EWMA	$\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1 - \lambda) r_{t-1}^2$	λ : Smoothing factor, controls sensitivity to new data.
ARIMA	$\Phi(B)(1 - B)^d y_t = \Theta(B) \epsilon_t$	p : AR order, d : Differencing order, q : MA order.
GARCH	$\sigma_t^2 = \omega + \sum_{i=1}^q \alpha_i r_{t-i}^2 + \sum_{j=1}^p \beta_j \sigma_{t-j}^2$	p : Order of lagged variances, q : Order of lagged squared returns, ω, α, β .

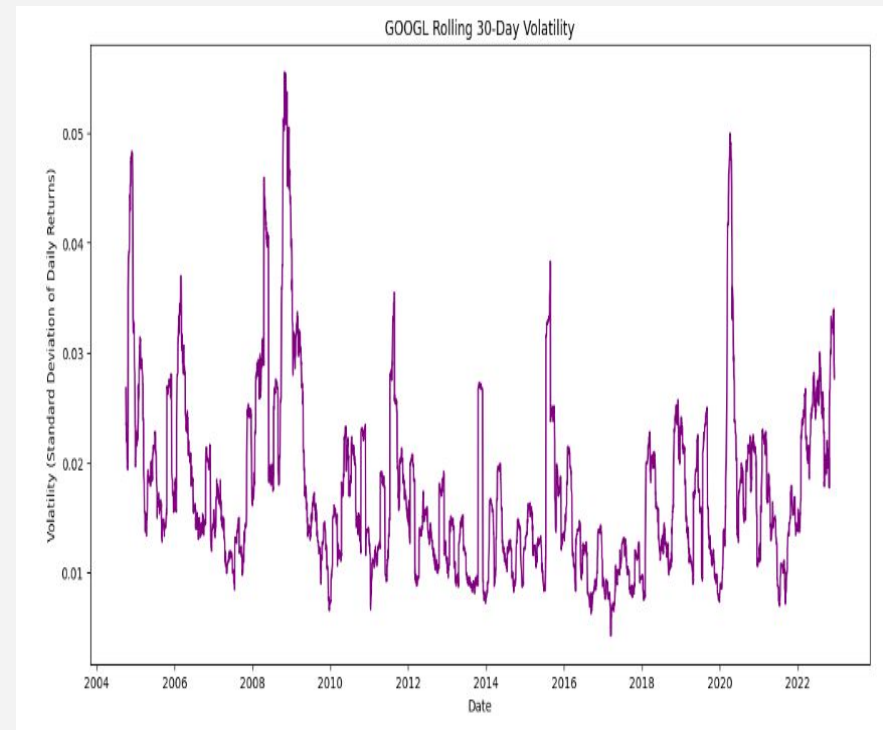
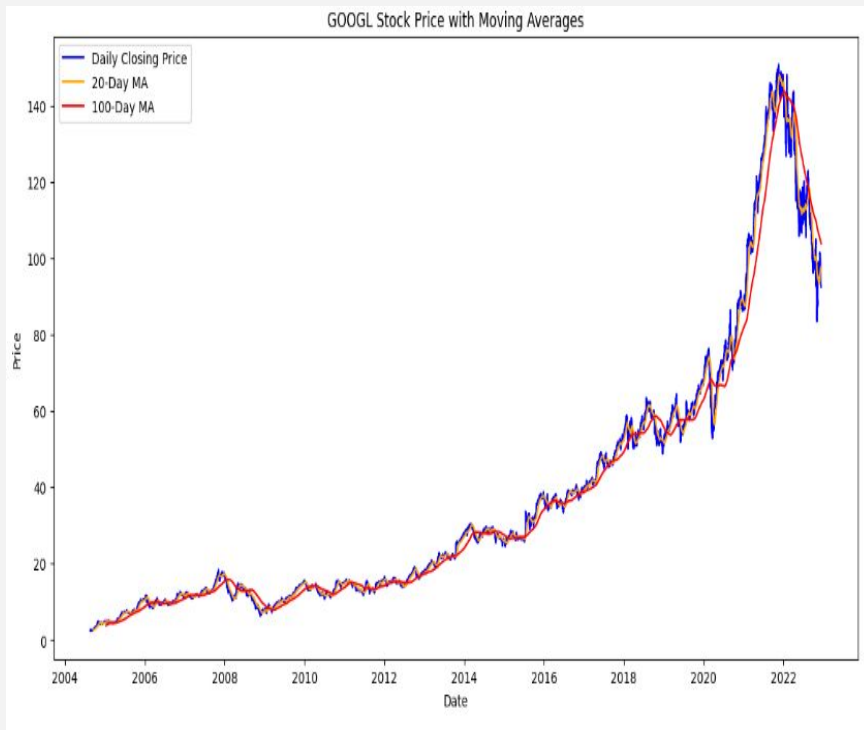
Shape-Constrained Regularization

- **Purpose:** Impose monotonicity in volatility forecasts to ensure alignment with theoretical market behavior.
- **Mechanism:**
 - Adds constraints to ensure that the forecasted variance at each horizon is non-decreasing
 - Mitigates overreaction to short-term noise & ensures smoother trends.
 - via: cvxpy library

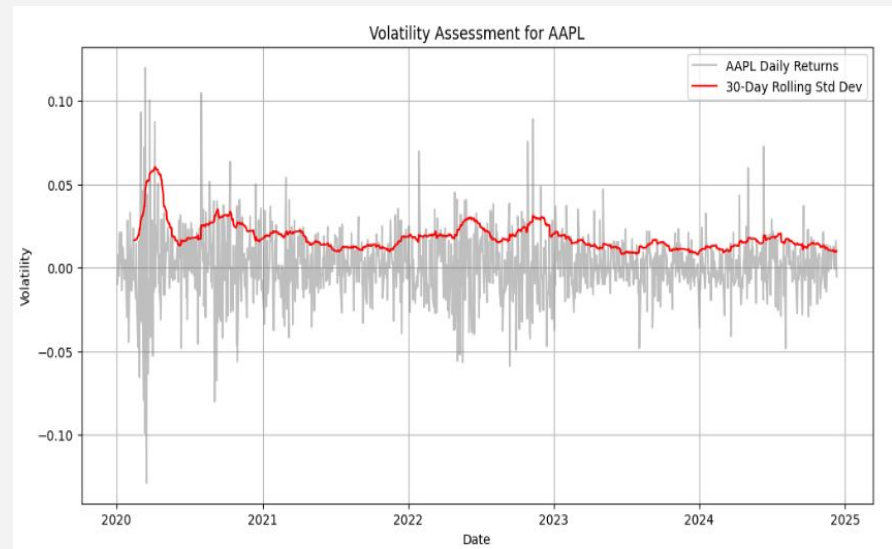
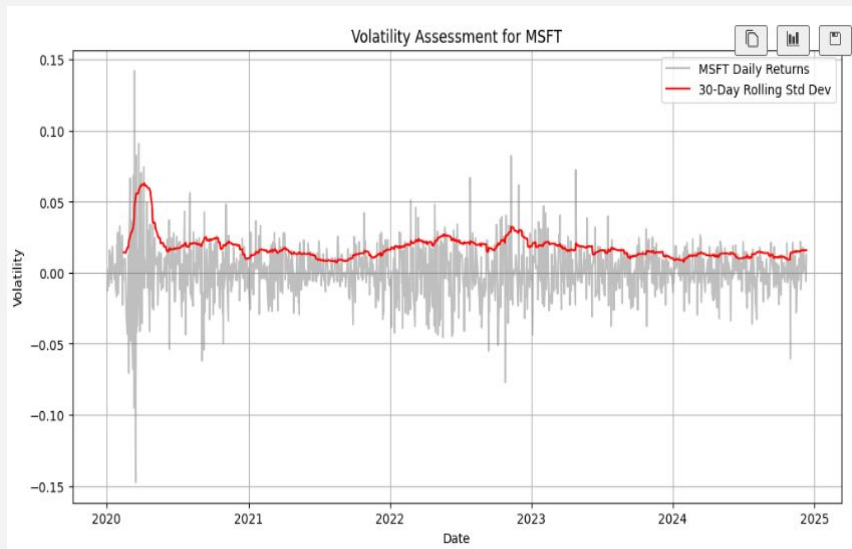
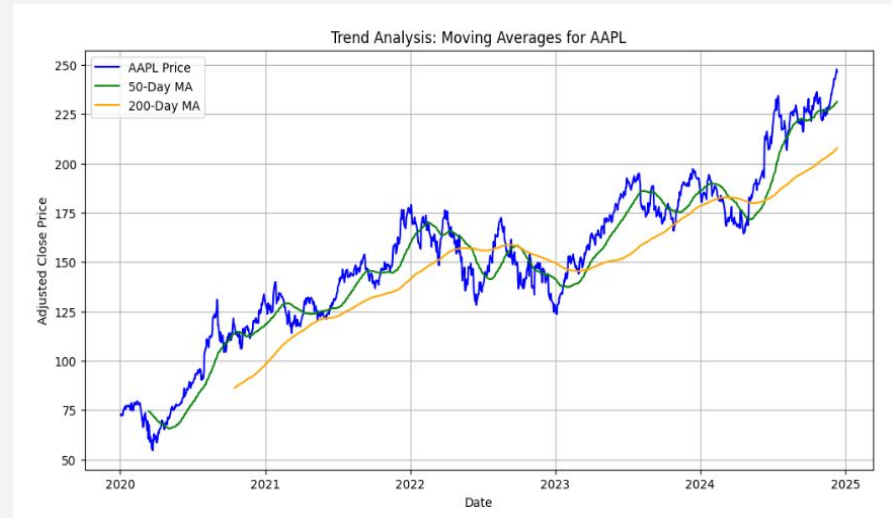
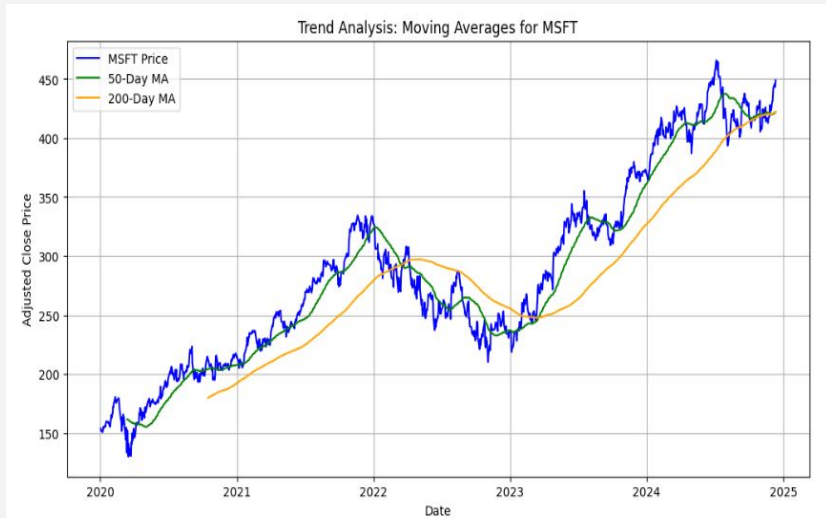
```
def apply_shape_constraints(forecasts):  
    n = len(forecasts)  
    constrained_forecasts = cp.Variable(n)  
    constraints = [constrained_forecasts[0] == forecasts[0]]  
  
    for t in range(1, n):  
        constraints.append(constrained_forecasts[t] >= constrained_forecasts[t-1]) # Monotonicity  
    objective = cp.Minimize(cp.sum_squares(constrained_forecasts - forecasts))  
  
    problem = cp.Problem(objective, constraints)  
    problem.solve()  
    return constrained_forecasts.value
```

EDA - Previously

- **EDA Objective:** Identify short-term and long-term trends, assess volatility patterns, and examine correlations.
- **Trend Analysis:** Moving averages to highlight short and long-term trends.
- **Volatility Assessment:** Rolling standard deviation of GOOGL's daily returns over 30 days.



EDA - Present



Implementation & Methodology

Models:

- EWMA: Optimized lambda using time-series cross-validation.
- ARIMA: Selected best order (p, d, q) using `auto_arima`.
- GARCH: Selected best (p, q) using AIC minimization.

Shape-Constrained Regularization:

- Enforced monotonicity on multi-horizon forecasts using `cvxpy`.
- Objective: Minimize the squared deviation from original forecasts under constraints.

Methodology

- **Forecast Horizon:** 5 days.
- **Evaluation Metrics:**
 - RMSE (Root Mean Squared Error).
 - MAE (Mean Absolute Error).
- **Comparison** between constrained & unconstrained forecasts.
- **Visual Analysis:** Historical volatility & forecast trends

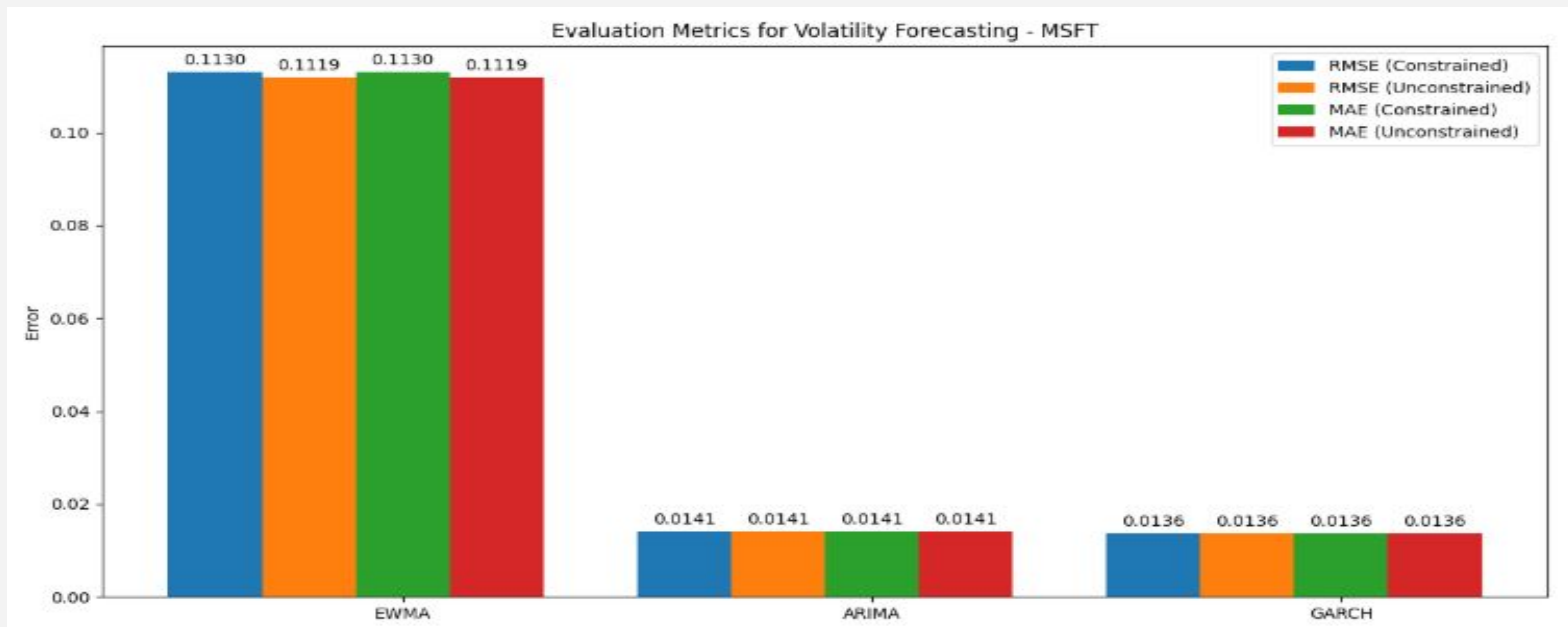
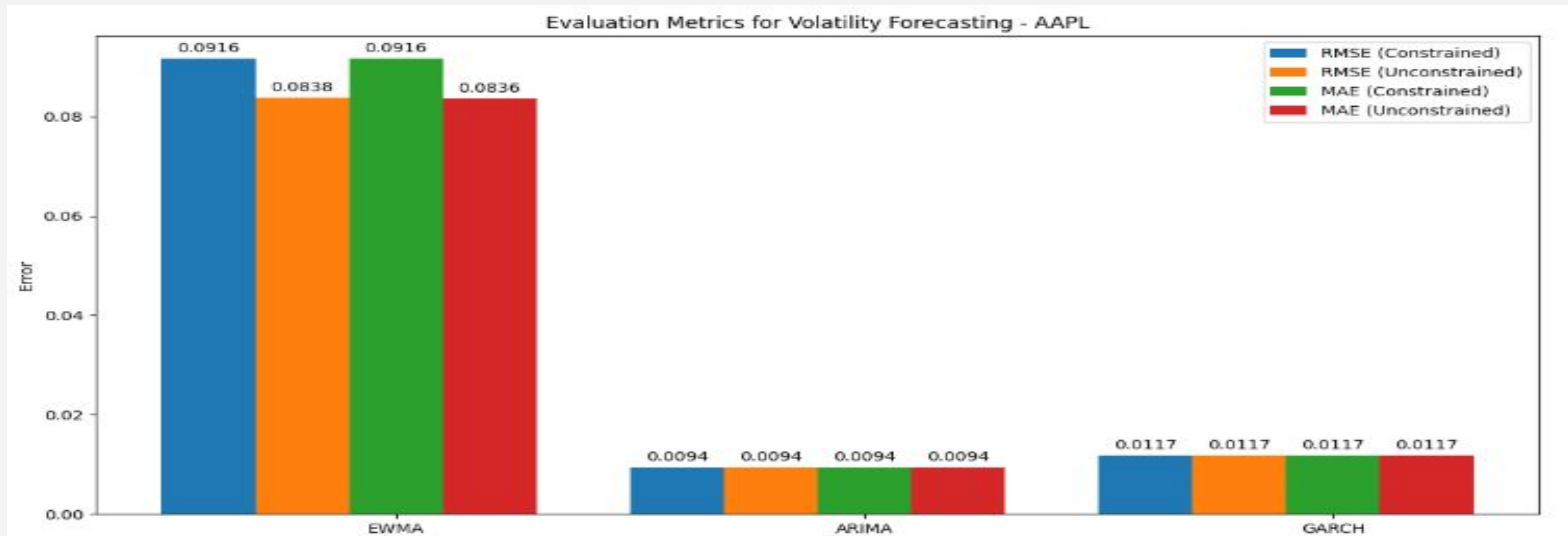
Code Snippets

```
# EWMA volatility calculation
def ewma_volatility(returns, lambd):
    vol = np.zeros_like(returns)
    vol[0] = returns.var()
    for t in range(1, len(returns)):
        vol[t] = lambd * vol[t-1] + (1 - lambd) * returns[t-1]**2
    return np.sqrt(vol)
```

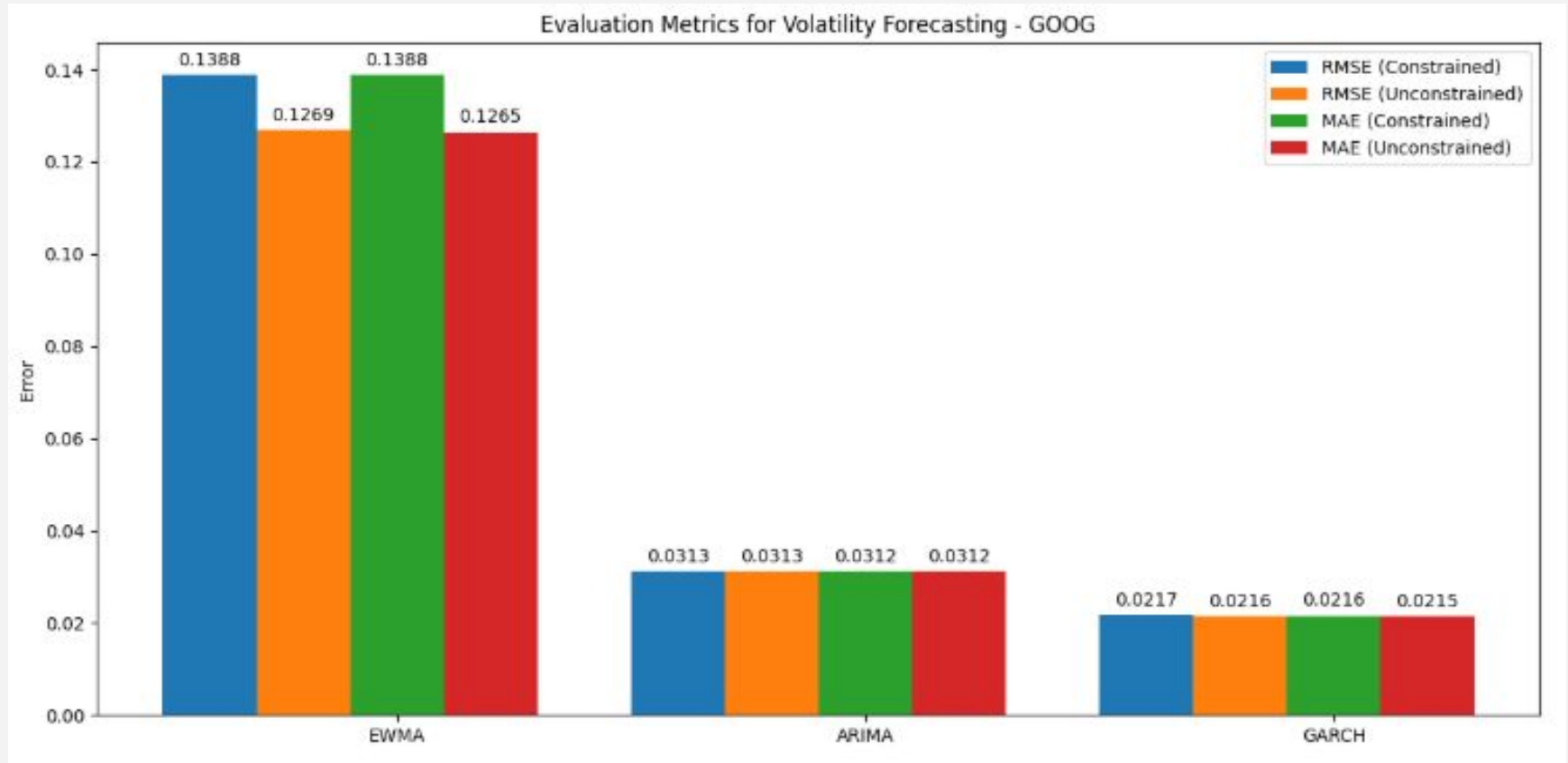
```
# Generate multi-horizon ARIMA forecasts
def arima_forecast(returns, order, horizon):
    model = ARIMA(returns**2, order=order).fit()
    forecast = model.forecast(steps=horizon)
    return np.sqrt(forecast)
```

```
# Generate multi-horizon GARCH forecasts
def garch_forecast(returns, p, q, horizon):
    model = arch_model(returns, vol='Garch', p=p, q=q).fit(dispen='off')
    forecast = model.forecast(horizon=horizon, reindex=True)
    return np.sqrt(forecast.variance.iloc[-1].values)
```


Results - Comparative Analysis



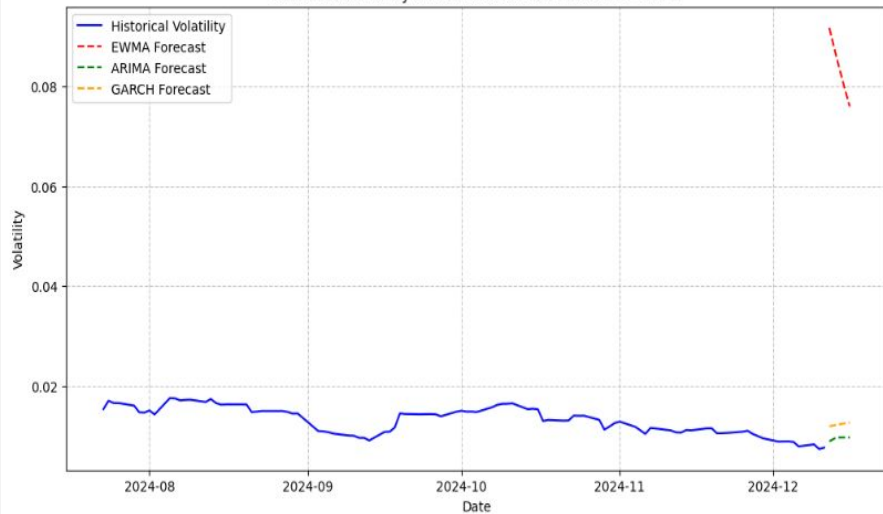
Results - Comparative Analysis



- Constrained forecasts increase RMSE/MAE slightly.
- GARCH and ARIMA outperform EWMA in terms of accuracy.

Historical Volatility & Forecasts

Historical Volatility and Unconstrained Forecasts - AAPL



Historical Volatility and Unconstrained Forecasts - MSFT



Historical Volatility and Unconstrained Forecasts - GOOG



Observations:

- GARCH and ARIMA closely track historical volatility.
- EWMA tends to overreact to recent changes.

Analysis & Interpretation

- **Impact of Monotonic Constraints:**
 - Slightly higher errors → constraints introduced unnecessary bias.
 - GARCH / ARIMA already produce smooth, realistic forecasts → reducing the need for additional constraints.
- **Model Performance:**
 - GARCH / ARIMA similar & better results compared to EWMA
 - EWMA is limited relative to the other two models

Conclusion

Key Takeaways:

- Monotonic constraints are not universally beneficial
 - should be applied cautiously
- GARCH / ARIMA models effectively capture volatility dynamics without constraints
- EWMA is more impacted by monotonic constraints (negatively in this case)

Practical Implications:

- For most scenarios, unconstrained forecasts are sufficient.
- Monotonic constraints may be useful in risk-averse or stress-testing applications.

Future Directions

- **Select** a time-period of higher stress & unpredictable
- **Alternative Constraints:**
 - Experiment with smoothness penalties instead of strict monotonicity.
- **Model Enhancements:**
 - Incorporate exogenous variables (e.g., macroeconomic indicators)
into ARIMA/GARCH models.